

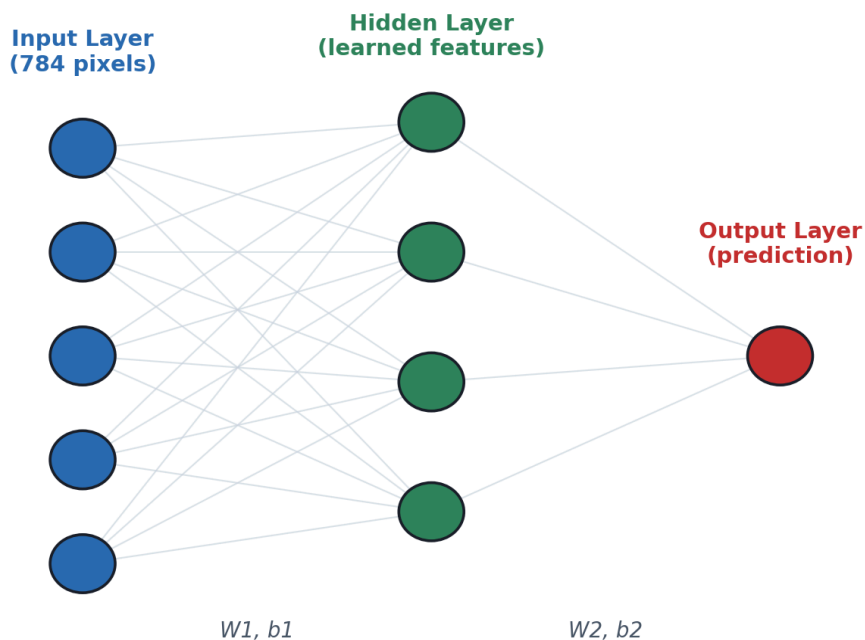
Deep Learning Crash Course

Neural networks, forward/backward propagation, and how training actually works

1. What Is a Neural Network?

A neural network is a stack of simple computations — neurons — arranged in layers, where each layer's output feeds into the next. Every neuron does the exact same basic thing: take a weighted sum of its inputs, add a bias, and pass the result through an activation function. What makes a network powerful isn't any single neuron; it's that stacking many simple neurons together lets the network build up increasingly complex, nonlinear patterns.

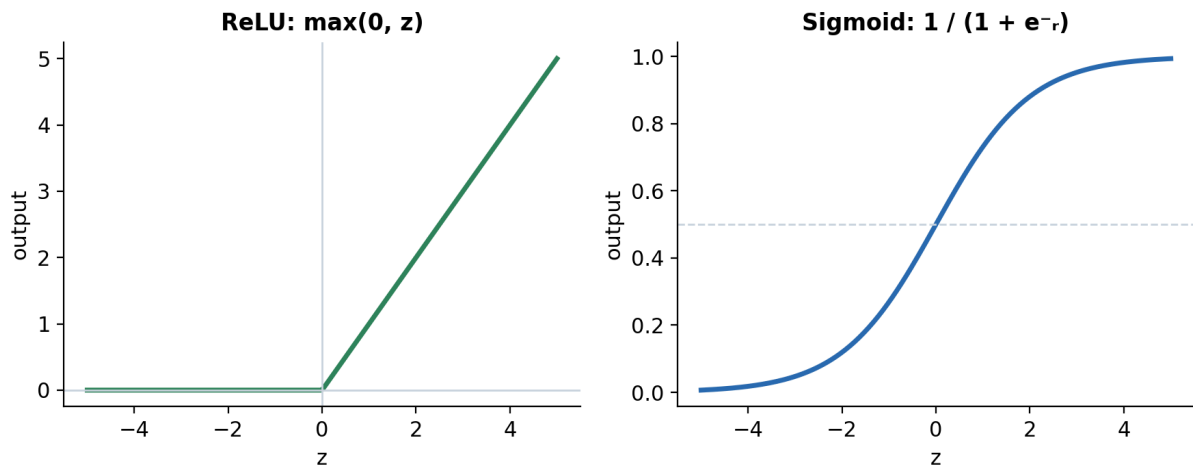
The key insight worth remembering: a single neuron with a sigmoid activation is mathematically identical to logistic regression. A neural network is what you get when you chain many of those together.



Input pixels flow through a hidden layer's learned weights ($W1, b1$), then an output layer's weights ($W2, b2$), producing a final prediction.

2. Layers

- **Input layer**



ReLU (left) is the standard choice for hidden layers — simple and fast. Sigmoid (right) squashes any input into a 0-1 probability, typically used in the output layer for binary classification.

- **ReLU** — Outputs the input directly if positive, otherwise outputs zero. Cheap to compute, and avoids a problem sigmoid has in deep networks (vanishing gradients). The default choice for hidden layers.
- **Sigmoid** — Squashes any real number into a 0-1 range — useful when you need the output interpreted as a probability, typically in the output layer for binary classification.
- **Softmax** — Generalizes sigmoid to multiple classes — outputs a full probability distribution across all classes, used in the output layer for multi-class classification.

4. Forward Propagation

Forward propagation is simply running your input data through the network, layer by layer, to produce a prediction. At each layer: take the weighted sum of the previous layer's output, add the bias, apply the activation function, pass the result to the next layer.

Forward Propagation



Backpropagation



Forward propagation computes a prediction and a loss. Backpropagation works in reverse, computing how much each layer's weights contributed to that loss.

The core mechanic, one layer at a time:

- 1. Start with how wrong the final output was (the gradient of the loss with respect to the output).
- 2. Use that to compute how much the output layer's weights should change.
- 3. Propagate the error backward into the hidden layer, by multiplying by the TRANSPOSE of the same weight matrix used going forward — this is the one genuinely new mechanic beyond logistic regression.
- 4. Use the hidden layer's activation derivative to figure out how much of that backward-propagated error actually passes through at each neuron.
- 5. Compute how much the hidden layer's weights should change.

Layer	A group of neurons that all receive the same inputs and compute in parallel. Input layer, hidden layer(s), output layer.
Activation function	A nonlinear function applied to a neuron's weighted sum. Without it, stacking layers would collapse into one linear function.
Forward propagation	Running data through the network, layer by layer, to produce a prediction.
Loss function	A number measuring how wrong the prediction was, compared to the true answer.
Backpropagation	Working backward from the loss to compute how much each weight contributed to the error, using the chain rule.
Gradient descent	Using those computed gradients to nudge every weight slightly in the direction that reduces loss.
Epoch	One full pass through the entire training dataset.

8. Where This Connects to Your Roadmap

Your from-scratch notebook builds exactly the architecture in Section 1's diagram: input → hidden layer (ReLU) → output (sigmoid), with binary cross-entropy as the loss — the same loss function from your logistic regression project, unchanged. The genuinely new exercise is deriving and implementing backpropagation through the hidden layer by hand, in raw NumPy, before ever letting a framework like PyTorch automate it. Once that's working, PyTorch's `.backward()` call should read as "automating the thing I already know how to do by hand," not as unexplained magic.